

1(a) Give **two** reasons why some programs are written in a low-level language.

1 .-----

2 .-----

[2]

(b) Describe the benefits of using a compiler instead of an interpreter when writing a program.

[3]

END OF QUESTION PAPER

Question		Answer/Indicative content	Marks	Guidance
1	a	1 mark each to max 2 • (machine code) does not need to be translated / compiled / interpreted • Direct control of hardware / memory • Faster execution time • Code can be optimised / shorter code / use less memory • Can program for specific hardware • Assembly language is fast to translate.	2 (AO1)	"More efficient" by itself is TV. Mark first answer on each line. BP6 relates to Assembly language being a one-to-one direct mapping to machine code. <u>Examiner's Comments</u> These questions assessed understanding of low-level languages and the use of compilers to translate from high-level code. Many candidates were able to do this well and clearly understood the need for high-level languages to be translated into machine code before the processor can execute this. The questions, however, did not ask just for this but specifically asked for reasons and benefits. Where candidates left their response as simply a discussion of the difference between high-level and low-level code or between translators and compilers, it was difficult for examiners to award marks. Better responses were able to give clear reasons for the use of low-level code (such as direct control of hardware and faster execution of code) and the benefits of using a compiler instead of an interpreter (such as being able to distribute an executable file with no access to source code and not needing to translate code again once this has been done).

Question		Answer/Indicative content	Marks	Guidance
	b	1 mark each to max 3 • Can produce an executable file • program/code runs/executes faster (than interpreted version) • end users do not need translator • Can be run again/multiple times without re-translating // only needs to translate once • End users have no access to source code // distributed with no source code... • ...cannot steal/copy/modify code/program • Shows all/multiple errors // shows errors at the end (of compilation) // creates error file • Compiler can optimise the code	3 (AO1)	Allow in reverse (e.g. "interpreter translates every time") Do not allow "no access to source code" unless clearly talking about end user. Allow if in context of distribution. Do not allow descriptions of how a compiler translates (e.g. "translates whole code in one go") "Faster / quicker" by itself is TV <u>Examiner's Comments</u> These questions assessed understanding of low-level languages and the use of compilers to translate from high-level code. Many candidates were able to do this well and clearly understood the need for high-level languages to be translated into machine code before the processor can execute this. The questions, however, did not ask just for this but specifically asked for reasons and benefits. Where candidates left their response as simply a discussion of the difference between high-level and low-level code or between translators and compilers, it was difficult for examiners to award marks. Better responses were able to give clear reasons for the use of low-level code (such as direct control of hardware and faster execution of code) and the benefits of using a compiler instead of an interpreter (such as being able to distribute an executable file with no access to source code and not needing to translate code again once this has been done).
		Total	5	